
LEARNING NIM STRATEGIES WITH BAYESIAN REINFORCEMENT LEARNING AND MCMC

Quyen Tran

Department of Mathematical Sciences
DePauw University
quyentran_2027@depauw.edu

ABSTRACT

Machine learning and reinforcement learning methods have achieved strong empirical performance across many applications, including game playing. However, this also raises concerns about the reliability, interpretability, and safety of model decisions, thereby motivating the study of learning algorithms in settings where the underlying problem structure is mathematically understood. Combinatorial games provide a useful setting for studying these questions because many admit mathematically characterized optimal strategies. Therefore, this paper uses the n -pile Nim game as a mathematically structured environment for studying Bayesian reinforcement learning through random-walk Metropolis Markov Chain Monte Carlo (MCMC) methods. The paper investigates how probabilistic posterior sampling over policy parameters influences the learned strategy and whether the resulting policy can recover the known combinatorial structure of optimal play. By combining theoretical analysis and empirical experiments, the paper shows that the proposed Bayesian reinforcement learning framework is able to recover the combinatorial structure underlying optimal play in the Nim game and achieve strong gameplay performance across multiple game settings. The code is available at: <https://github.com/quyenlhtran/bayesian-rl-nim>.

1 INTRODUCTION

Machine learning (ML) is a broad field concerned with developing algorithms that learn patterns from data and use those patterns to make predictions or decisions [15]. Over the past decade, methods such as deep neural networks [13], statistical learning techniques [3], and reinforcement learning [23] have achieved strong empirical performance across a wide range of applications, including computer vision [9, 12], natural language processing [24], recommendation systems [11], and game playing [20]. Many of these approaches can also be studied from a statistical perspective, where learning is viewed as an inference problem based on observed data. In particular, Bayesian machine learning represents uncertainty over model parameters through probability distributions and updates these beliefs using observed data [16]. When combined with reinforcement learning, this leads to Bayesian reinforcement learning frameworks that model uncertainty in sequential decision problems [8]. However, in many cases, the resulting posterior distributions cannot be computed analytically, requiring approximation methods such as variational inference [10], particle-based methods [21], and Markov Chain Monte Carlo (MCMC) [17].

Although modern machine learning models have achieved strong empirical performance in many applications, they are often evaluated primarily based on predictive accuracy [6]. As models become increasingly complex, their internal computations are often treated as black boxes, making it difficult to understand how particular predictions are produced [7]. This lack of transparency raises concerns about the reliability, interpretability, and safety of model decisions, particularly in settings where incorrect predictions may have significant consequences [19]. Recent work has attempted to address this issue through various interpretability methods, including feature attribution [14, 22], probing methods [1, 5], and model visualization techniques [25]. However, understanding the mechanisms through which learning algorithms arrive at their decisions remains challenging when the true structure

of the problem is unknown. This difficulty motivates the study of learning algorithms in controlled environments where the underlying structure of the problem is known.

Combinatorial games provide a natural setting for this type of investigation because many of them have well-understood mathematical solutions and a structured space of game configurations [2]. In particular, the Nim game with multiple piles has a known optimal strategy that can be characterized analytically [4]. This makes it possible to compare the behavior of a learning algorithm with the mathematically optimal solution and to examine whether the algorithm recovers the underlying strategic structure of the game.

Motivated by these observations, this paper makes the following contributions:

1. **A theoretical analysis of Bayesian reinforcement learning and MCMC-based sampling in combinatorial game playing.** The paper develops a mathematical analysis of both the combinatorial structure of Nim and the associated Metropolis Markov chain, including the characterization of optimal play together with invariance and convergence properties of the sampling procedure. This provides a theoretical connection between probabilistic posterior sampling, reinforcement learning, and the empirical behavior of the learned policy.
2. **A Bayesian reinforcement learning framework using random-walk Metropolis MCMC for the n -pile Nim game.** The paper introduces one approach for incorporating MCMC-based Bayesian inference into reinforcement learning and combinatorial game playing by approximating posterior distributions over policy parameters through random-walk Metropolis sampling.
3. **An empirical study of policy learning in Nim.** Using experiments on both two-pile and general n -pile Nim, the paper shows that the learned policy strongly prioritizes moves to nim-sum-zero positions and achieves near-optimal performance across multiple values of n . This demonstrates that the probabilistic learning framework is able to recover the theoretically characterized combinatorial structure of optimal play in Nim.

2 PRELIMINARIES

2.1 THE GAME OF NIM

The n -pile Nim game provides a mathematically structured setting with an explicitly characterized optimal strategy, making it a good choice for studying Bayesian reinforcement learning in a well-controlled environment. Players take turns selecting one pile and removing a positive number of objects from it, and the player who removes the last object wins. Figure 1 illustrates the game-state representation and an example legal move.

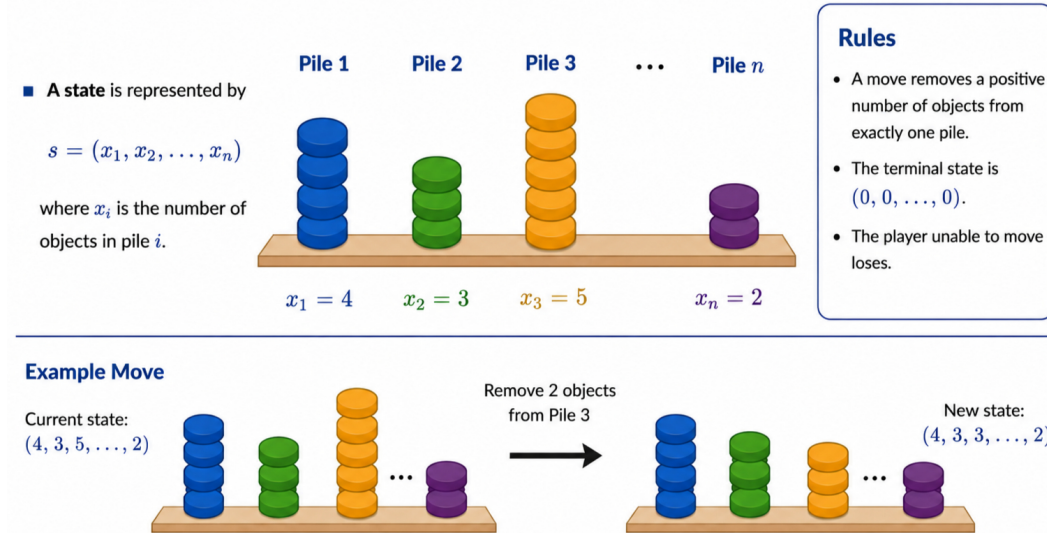


Figure 1: Illustration of the n -pile Nim game and a legal move transition.

To characterize optimal play, the paper introduces a formal description of the game in terms of states, moves, and position types.

Definition 2.1 (State). A state of the game is represented by $s = (x_1, x_2, \dots, x_n)$, where $x_i \in \mathbb{Z}_{\geq 0}$ denotes the number of objects in pile i .

Definition 2.2 (Legal Move). A legal move consists of selecting a pile i with $x_i > 0$ and removing a positive number $k \leq x_i$ of objects from that pile. The resulting state is $(x_1, \dots, x_i - k, \dots, x_n)$.

Definition 2.3 (Terminal State). A state is terminal if it is of the form $(0, 0, \dots, 0)$. At such a state, no legal moves are available, and the player to move loses.

Theorem 2.4 (Base- b Expansion [18]). *Let b be an integer with $b \geq 2$. Then every positive integer n can be written uniquely in the form*

$$n = a_k b^k + a_{k-1} b^{k-1} + \dots + a_1 b + a_0,$$

where $k \in \mathbb{Z}_{\geq 0}$, $0 \leq a_i < b$ for all i , and $a_k \neq 0$.

Definition 2.5 (Binary Expansion). By specializing Theorem 2.4 to $b = 2$, every nonnegative integer $a \in \mathbb{Z}_{\geq 0}$ has a unique representation

$$a = \sum_{k=0}^m a_k 2^k, \quad a_k \in \{0, 1\}, \quad a_m = 1,$$

called the binary expansion of a .

Definition 2.6 (Binary Operation $\oplus$). Let $a, b \in \mathbb{Z}_{\geq 0}$ with binary expansions

$$a = \sum_{k=0}^m a_k 2^k, \quad b = \sum_{k=0}^m b_k 2^k,$$

where $a_k, b_k \in \{0, 1\}$. The binary operation $\oplus$ is defined by

$$a \oplus b := \sum_{k=0}^m (a_k + b_k \bmod 2) 2^k.$$

By the uniqueness of binary expansion, the operation $\oplus$ is well-defined. Moreover, since it is defined digit-wise via addition modulo 2, $\oplus$ is associative and commutative. Hence, we may define the nim-sum of a state as follows.

Definition 2.7 (Nim-Sum). For a state $s = (x_1, \dots, x_n)$, the nim-sum is defined as

$$x_1 \oplus x_2 \oplus \dots \oplus x_n.$$

Theorem 2.8 (Nim Characterization). *A state $s = (x_1, \dots, x_n)$ is a losing position if and only if $x_1 \oplus x_2 \oplus \dots \oplus x_n = 0$. Otherwise, the state is a winning position.*

Sketch of proof. If the nim-sum is zero, any legal move changes one pile and results in a nonzero nim-sum, so no move leads to another losing position. Conversely, if the nim-sum is nonzero, one can reduce a suitable pile so that the resulting nim-sum becomes zero. A detailed proof is given in Appendix A.1. $\square$

In the special case of two piles, a state is given by (a, b) , and the nim-sum reduces to $a \oplus b$.

The condition $a \oplus b = 0$ holds if and only if $a = b$. Therefore, a two-pile state (a, b) is a losing position if and only if the two piles have equal size. Equivalently, a two-pile state is winning if and only if the two piles have different sizes. In that case, the winning move is to reduce the larger pile so that the two piles become equal.

2.2 MATHEMATICAL FRAMEWORK FOR BAYESIAN REINFORCEMENT LEARNING

The Nim game defined in the Subsection 2.1 can be viewed as a sequential decision problem. At time t , the system is in a game state $s_t \in \mathcal{S}$, where $\mathcal{S}$ denotes the set of all possible game configurations. In our framework, states are represented in canonical form by sorting the pile sizes in nondecreasing order, thereby identifying equivalent pile configurations.

Definition 2.9 (Canonical State Representation). For a state $s = (x_1, \dots, x_n)$, its canonical representation is the reordered state $\text{can}(s) = (x_{(1)}, \dots, x_{(n)})$, where $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$.

Definition 2.10 (Action). Given a state $s = (x_1, \dots, x_n)$, an *action* consists of selecting a pile i with $x_i > 0$ and removing a positive number $k \leq x_i$ of objects from that pile. The set of admissible actions is

$$\mathcal{A}(s) = \{(i, k) : 1 \leq i \leq n, 1 \leq k \leq x_i\}.$$

An action (i, k) produces the state $(x_1, \dots, x_i - k, \dots, x_n)$.

Definition 2.11 (Transition Function). The transition function $T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ maps a state and an action to the next state by $T((x_1, \dots, x_n), (i, k)) = \text{can}(x_1, \dots, x_i - k, \dots, x_n)$.

Definition 2.12 (Optimal Action). Let $s \in \mathcal{S}$ be nonterminal, and let $a \in \mathcal{A}(s)$. We say that a is an *optimal action* if the resulting state has nim-sum zero, that is,

$$x'_1 \oplus x'_2 \oplus \dots \oplus x'_n = 0,$$

where

$$T(s, a) = (x'_1, \dots, x'_n).$$

Definition 2.13 (Feature Vector). Let $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. A feature vector is a mapping $\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^d$, for some fixed dimension d , where $\phi(s, a)$ encodes structural properties of the state-action pair.

Definition 2.14 (Parameterized Policy). Let $\theta \in \mathbb{R}^d$ denote a parameter vector, with coordinates $\theta = (\theta_1, \dots, \theta_d)$. We define a linear scoring function

$$f_\theta(s, a) = \theta^\top \phi(s, a).$$

The policy $\pi_\theta(a | s)$ is given by

$$\pi_\theta(a | s) = \frac{e^{f_\theta(s, a)}}{\sum_{a' \in \mathcal{A}(s)} e^{f_\theta(s, a')}}.$$

Definition 2.15 (Greedy Policy). Given $\theta \in \mathbb{R}^d$, a greedy action in state s is any action $a^* \in \arg \max_{a \in \mathcal{A}(s)} f_\theta(s, a)$.

Definition 2.16 (Reward). Let (s_t, a_t) be a state-action pair selected at time t during an episode. The reward is defined by

$$r_t = \begin{cases} 1, & \text{if the terminal state is reached by the current player,} \\ -1, & \text{otherwise.} \end{cases}$$

Definition 2.17 (Transition Sample). A transition sample is a tuple (s, a, r) , where $s \in \mathcal{S}$ is a state, $a \in \mathcal{A}(s)$ is an admissible action, and $r \in \{-1, 1\}$ is the associated reward.

Definition 2.18 (Dataset). A dataset is a finite collection of transition samples $D = \{(s_i, a_i, r_i)\}_{i=1}^N$.

Definition 2.19 (Prior Distribution). A prior distribution $p(\theta)$ represents the initial belief about the parameter $\theta \in \mathbb{R}^d$ before observing any data. In this work, we assume a Gaussian prior $\theta \sim N(0, \sigma^2 I_d)$, where $\sigma^2 > 0$ is a fixed variance parameter and I_d is the $d \times d$ identity matrix.

Definition 2.20 (Likelihood). Given a dataset $D = \{(s_i, a_i, r_i)\}_{i=1}^N$, the likelihood is defined by

$$p(D | \theta) = \prod_{i=1}^N \pi_\theta(a_i | s_i)^{\lambda r_i},$$

where $\lambda > 0$ is a scaling parameter.

Definition 2.21 (Posterior Distribution). The posterior distribution of θ given the data D is defined by

$$p(\theta | D) = \frac{p(D | \theta)p(\theta)}{\int p(D | \theta')p(\theta') d\theta'}.$$

There are two distinct notions of “state” in this work. The first refers to game states $s \in \mathcal{S}$, which describe pile configurations in Nim. The second refers to the states of the Metropolis Markov chain, namely parameter vectors $\theta \in \mathbb{R}^d$. The reinforcement learning process evolves over game states, while the MCMC algorithm evolves over the parameter space.

2.3 MARKOV CHAIN MONTE CARLO

The posterior distribution $p(\theta \mid D)$ defined in the previous subsection does not admit a closed-form expression, as the normalizing constant in Bayes' rule is intractable. To approximate expectations with respect to this distribution, we construct a Markov chain $\{\theta^{(t)}\}_{t \geq 0}$ on the parameter space $\mathbb{R}^d$, where each state of the chain corresponds to a parameter vector and transitions are generated by proposing a new parameter based on the current state and accepting or rejecting it according to the posterior distribution.

Definition 2.22 (Random-Walk Metropolis Algorithm). The Markov chain is the sequence $\{\theta^{(t)}\}_{t \geq 0}$ in $\mathbb{R}^d$ defined as follows. Given the current state $\theta^{(t)}$, a proposal θ' is generated by

$$\theta' = \theta^{(t)} + \eta, \quad \eta \sim N(0, \tau^2 I_d),$$

where $\tau > 0$ is a tuning parameter controlling the proposal scale and I_d is the $d \times d$ identity matrix.

The next state $\theta^{(t+1)}$ is defined by

$$\theta^{(t+1)} = \begin{cases} \theta', & \text{with probability } \min \left\{ 1, \frac{p(\theta' \mid D)}{p(\theta^{(t)} \mid D)} \right\}, \\ \theta^{(t)}, & \text{otherwise.} \end{cases}$$

In our implementation, the algorithm is initialized at a fixed parameter value and runs for a predetermined number of iterations. After discarding an initial portion of the chain to reduce the influence of the starting point (burn-in), we retain samples at regular intervals to reduce dependence between successive iterates (thinning).

Definition 2.23 (Empirical Posterior Mean). Let $\{\theta^{(m)}\}_{m=1}^M$ denote the retained samples after burn-in and thinning. The empirical posterior mean is defined by

$$\bar{\theta} = \frac{1}{M} \sum_{m=1}^M \theta^{(m)}.$$

The parameter $\bar{\theta}$ is used to define a policy through the scoring function $f_{\bar{\theta}}(s, a) = \bar{\theta}^\top \phi(s, a)$, and serves as a representative estimate of the posterior distribution.

3 THEORETICAL ANALYSIS

In this section, we analyze the behavior of the Markov chain constructed in the previous section and its implications for learning a policy for the Nim game.

3.1 INVARIANT DISTRIBUTION

Proposition 3.1. *The Markov chain $\{\theta^{(t)}\}_{t \geq 0}$ defined by the random-walk Metropolis algorithm has invariant distribution $p(\theta \mid D)$.*

Sketch of proof. The proposal distribution is symmetric, i.e., $q(\theta, \theta') = q(\theta', \theta)$.

Using this symmetry and the definition of the Metropolis acceptance probability, we can verify the detailed balance condition

$$p(\theta \mid D) K(\theta, \theta') = p(\theta' \mid D) K(\theta', \theta).$$

It follows that $p(\theta \mid D)$ is invariant. The full proof is given in Appendix A.2. □

3.2 CONVERGENCE OF EMPIRICAL AVERAGES

We now justify the use of empirical averages computed from the Metropolis chain.

Proposition 3.2 (Irreducibility). *Assume that $p(\theta \mid D) > 0$ for all $\theta \in \mathbb{R}^d$. Then the random-walk Metropolis chain $\{\theta^{(t)}\}_{t \geq 0}$ is irreducible.*

Sketch of proof. The Gaussian proposal density is strictly positive everywhere, and the acceptance probability is also strictly positive under the assumption $p(\theta \mid D) > 0$. Hence, from any state, the chain can move to any region with positive probability. A detailed proof is provided in Appendix A.3. $\square$

Proposition 3.3 (Aperiodicity). *Assume that $p(\theta \mid D) > 0$ for all $\theta \in \mathbb{R}^d$. Then the random-walk Metropolis chain $\{\theta^{(t)}\}_{t \geq 0}$ is aperiodic.*

Sketch of proof. There is a positive probability that a proposed move is rejected, in which case the chain remains at the current state. Hence, the chain has a self-transition with positive probability, implying period 1. A detailed proof is provided in Appendix A.4. $\square$

By Propositions 3.1–3.3, the required conditions for convergence are satisfied by the random-walk Metropolis chain. We therefore state the following convergence result.

Theorem 3.4 (Convergence of empirical averages). *Suppose that the random-walk Metropolis chain $\{\theta^{(t)}\}_{t \geq 0}$ has invariant distribution $p(\theta \mid D)$, and that the chain is irreducible and aperiodic. Let $h : \mathbb{R}^d \rightarrow \mathbb{R}$ be such that the posterior expectation*

$$\int_{\mathbb{R}^d} |h(\theta)| p(\theta \mid D) d\theta$$

is finite. Then

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T h(\theta^{(t)}) = \int_{\mathbb{R}^d} h(\theta) p(\theta \mid D) d\theta.$$

Proof. Under the stated assumptions, the Markov chain is irreducible, aperiodic, and admits $p(\theta \mid D)$ as an invariant distribution. Hence, the chain is ergodic. The result then follows from the Ergodic Theorem. $\square$

Corollary 3.5 (Posterior mean parameter estimation). *Under the assumptions of Theorem 3.4, if*

$$\int_{\mathbb{R}^d} \|\theta\| p(\theta \mid D) d\theta < \infty,$$

then

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \theta^{(t)} = \int_{\mathbb{R}^d} \theta p(\theta \mid D) d\theta =: \bar{\theta}.$$

Sketch of proof. Apply Theorem 3.4 to the coordinate functions $h_i(\theta) = \theta_i$ for $i = 1, \dots, d$, and combine the resulting limits to obtain the vector convergence. A detailed proof is given in Appendix A.5. $\square$

Implication for policy learning. The results above imply that posterior expectations can be approximated by empirical averages along the Metropolis chain. Therefore, quantities used to define the policy, such as the parameter vector or action scores, can be estimated directly from the sampled sequence $\{\theta^{(t)}\}$.

In practice, we approximate the posterior mean parameter

$$\bar{\theta} = \int_{\mathbb{R}^d} \theta p(\theta \mid D) d\theta$$

using the empirical average of the Metropolis samples after burn-in and thinning.

The learned policy is then defined by

$$\pi(s) = \arg \max_{a \in \mathcal{A}(s)} \bar{\theta}^\top \phi(s, a),$$

which selects the action with the highest posterior mean score.

4 EXPERIMENTS

This section evaluates whether the proposed Bayesian reinforcement learning framework is able to recover the optimal strategy for the Nim game. In addition to evaluating gameplay performance, the experiments investigate the effect of feature representations, prior distributions, scalability with respect to the number of piles, and comparisons with several baseline methods.

4.1 EXPERIMENTAL SETUP

Problem setting. The experiments are conducted on both the two-pile and the general n -pile versions of the Nim game. In the case $n = 2$, a state is of the form (a, b) , and optimal play is characterized by moving to a state with $a = b$. In the general case, a state is given by $s = (x_1, \dots, x_n)$, and losing positions are characterized by the condition $x_1 \oplus \dots \oplus x_n = 0$. To study the effect of problem size, experiments are performed for multiple values of n (e.g., $n \in \{2, 4, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$), with initial states generated randomly and represented in canonical form. The maximum initial pile size is fixed across experiments so that changes in performance and runtime primarily reflect the effect of increasing the number of piles.

Data generation. A dataset is constructed by simulating gameplay episodes between a parameterized agent following the policy π_θ and a random opponent that selects legal moves uniformly. Each episode produces transitions (s_t, a_t) , and a reward $r_t \in \{-1, 1\}$ is assigned according to the final outcome of the game. The dataset is defined as $D = \{(s_i, a_i, r_i)\}_{i=1}^N$, and in the implementation 4,000 episodes are simulated.

Feature representation. Each state-action pair (s, a) is mapped to a feature vector $\phi(s, a) \in \mathbb{R}^{10}$ that encodes structural properties of the game. Let $T(s, a) = s'$ denote the resulting state after applying an action $a = (i, k)$. The feature vector includes a bias term, a component indicating whether the resulting state has nim-sum equal to zero, and additional components capturing terminal states, reduction in the total number of objects, the magnitude of the move, game progress, whether the selected pile is maximal, and normalized quantities related to the nim-sum before and after the move. Since the nim-sum-zero feature directly encodes known combinatorial structure, additional feature-ablation experiments are conducted to evaluate how strongly the learned policy depends on this feature.

Prior distributions. To study the effect of prior assumptions, several prior distributions over the policy parameter vector θ are considered. These include zero-centered Gaussian priors, shifted Gaussian priors, wider Gaussian priors, half-normal priors, and truncated normal priors. The experiments investigate whether the learned policy is sensitive to the prior specification or whether the gameplay data dominate the posterior behavior.

Posterior inference. The posterior distribution $p(\theta \mid D)$ is approximated using the random-walk Metropolis algorithm described in Section 2.2. For each configuration, the sampler is run for 4,000 iterations with a burn-in period of 1,000 iterations and thinning interval 10. Multiple chains are executed in parallel to improve computational efficiency. From the retained samples $\{\theta^{(m)}\}_{m=1}^M$, the empirical posterior mean $\bar{\theta} = \frac{1}{M} \sum_{m=1}^M \theta^{(m)}$ is used to define the final greedy policy.

Evaluation protocol. For each value of n , the parameter $\bar{\theta}$ defines a scoring function $f_{\bar{\theta}}(s, a) = \bar{\theta}^\top \phi(s, a)$, and the corresponding greedy policy is evaluated by simulating 1,000 games against a random opponent. Performance is measured using win rate, optimal-action accuracy, and runtime. Optimal-action accuracy is defined as the proportion of evaluated states in which the selected action produces a state with nim-sum equal to zero whenever such an action exists.

Baseline methods. The proposed Bayesian reinforcement learning framework is compared with several baseline methods, including a random policy, a greedy heuristic policy, tabular Q-learning, minimax search, and alpha-beta pruning. All methods are evaluated using the same gameplay environments and evaluation metrics to compare strategic performance and computational cost. These comparisons are intended to evaluate both the strategic quality and computational scalability of the

proposed Bayesian approach relative to classical search-based and reinforcement-learning-based methods.

Implementation details. All experiments were implemented in Python and executed in a Google Colab environment with a GPU-backed runtime (G4 option), using an NVIDIA RTX PRO 6000 Blackwell GPU. The GPU was primarily used to accelerate batched Metropolis updates. NumPy and PyTorch were used for computation.

4.2 RESULTS

As shown in Figure 2, the experimental results demonstrate that the proposed Bayesian reinforcement learning framework achieves near-optimal gameplay performance comparable to classical optimal-play methods while maintaining strong computational scalability and overall efficiency across different n -pile Nim settings.

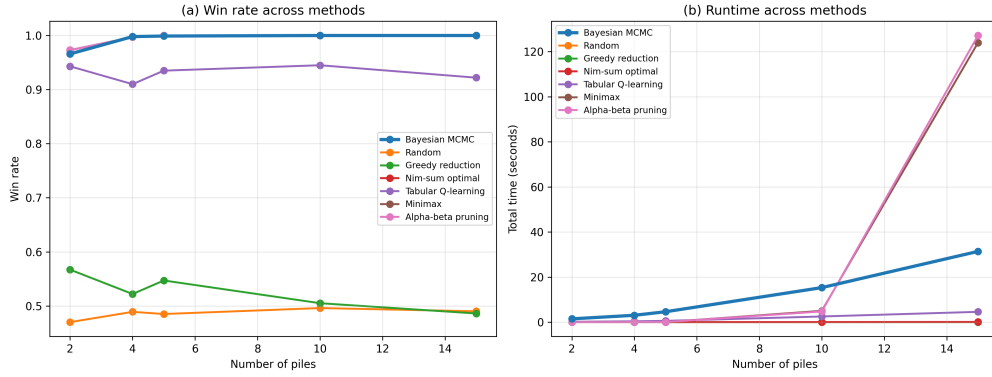


Figure 2: Baseline comparison between the proposed Bayesian reinforcement learning method and other gameplay strategies.

The proposed Bayesian MCMC approach consistently achieved near-perfect gameplay performance across all tested settings, with win rates essentially matching the theoretical nim-sum optimal strategy. Classical search-based approaches such as minimax and alpha-beta pruning also achieved optimal gameplay performance, but their runtime increased rapidly as the number of piles grew, making them substantially more computationally expensive for larger problem sizes. Tabular Q-learning achieved relatively strong gameplay performance and remained competitive with the proposed Bayesian MCMC framework for smaller problem sizes, but its performance remained consistently lower and gradually degraded as the number of piles increased. Random play and greedy reduction achieved the lowest computational cost among all compared methods, but their gameplay performance remained substantially weaker, with win rates close to 50%. Therefore, the proposed Bayesian MCMC framework achieved the strongest overall balance between gameplay performance and computational efficiency across the tested n -pile Nim settings.

To better understand the behavior of the proposed framework, the effects of feature representations and prior distributions are further investigated, followed by an analysis of the learned policy behavior.

Feature Ablation. To investigate the importance of the feature representation, the proposed Bayesian reinforcement learning framework was evaluated using three different feature sets: the full feature representation, a feature set with the exact nim-sum indicator removed, and a smaller reduced feature subset. Figure 3 shows the resulting gameplay performance, optimal-action accuracy, and computational runtime across different numbers of piles.

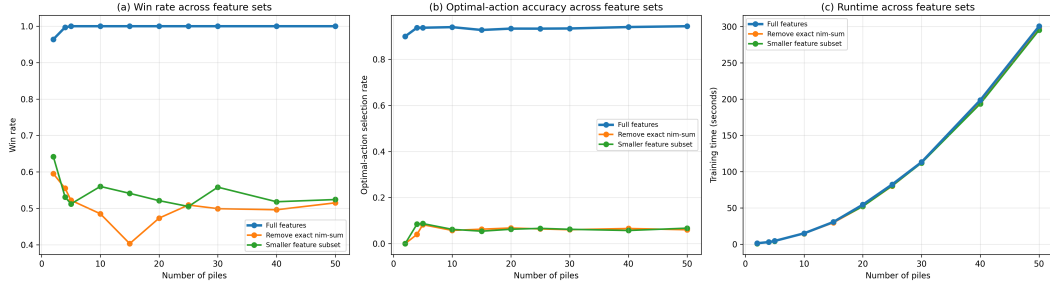


Figure 3: Feature-ablation results for the proposed Bayesian reinforcement learning method across different values of n .

The full feature representation consistently achieved near-perfect gameplay performance across all tested settings, with win rates close to 100% and optimal-action selection accuracy above 90%. In contrast, removing the exact nim-sum feature caused a substantial performance degradation, with win rates decreasing to approximately 40%–70% and optimal-action selection accuracy dropping below 10% across most values of n . Similar behavior was observed for the smaller feature subset, which also failed to recover the optimal strategy reliably.

These results suggest that incorporating nim-sum-related combinatorial information is essential for learning the underlying optimal structure of the Nim game. Without these features, the learned policy struggles to identify the theoretically optimal actions despite extensive gameplay experience. At the same time, the runtime behavior remained similar across all feature sets, indicating that the performance differences primarily arise from the representational quality of the features rather than computational cost.

Prior Distribution Analysis. To investigate the sensitivity of the proposed framework to the prior specification, the experiments were repeated using several different prior distributions, including Gaussian priors with different means and variances, a half-normal prior, and a truncated normal prior. Figure 4 summarizes the resulting gameplay performance, optimal-action accuracy, and computational runtime across different numbers of piles.



Figure 4: Effect of different prior distributions on the learned-policy behavior and gameplay performance of the proposed Bayesian reinforcement learning framework across the n -pile Nim setting.

Overall, the results show that the proposed Bayesian reinforcement learning framework is relatively robust to the choice of prior distribution. Most prior configurations achieved nearly identical performance, with win rates close to 100% and optimal-action selection accuracy consistently around 93%–94% across nearly all tested values of n . The computational runtime also remained highly similar across different priors.

The truncated normal prior exhibited slightly less stable behavior for some intermediate values of n , where both the win rate and optimal-action accuracy temporarily decreased compared to the other priors. However, the overall differences across prior distributions remained relatively small. These results suggest that the learned policy is primarily driven by the gameplay data and feature representation rather than strong dependence on the initial prior assumptions.

Learned Policy Analysis. To further investigate the behavior of the learned policy, the posterior mean parameters and the resulting action-selection behavior were analyzed for the two-pile Nim setting because the full state space can be visualized directly and compared against the theoretical nim-sum characterization of optimal play. Since the framework was trained under multiple feature representations and prior distributions, the model selected for this analysis corresponds to the configuration that demonstrated the strongest and most consistent performance across different values of n , namely the full feature representation with a Gaussian prior distribution having mean 0 and standard deviation 10.

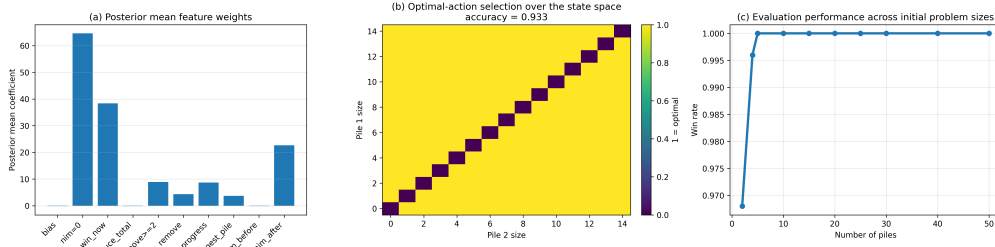


Figure 5: Learned-policy behavior for the two-pile Nim setting, including posterior feature weights and optimal-action structure.

Figure 5(a) shows the posterior mean feature coefficients learned by the Bayesian reinforcement learning framework. The largest coefficient was assigned to the feature indicating whether the resulting state has nim-sum equal to zero, suggesting that the model strongly prioritizes transitions toward theoretically losing states for the opponent. Other strategically relevant features, such as move size and progress-related features, also received positive weights but with substantially smaller magnitudes.

Figure 5(b) visualizes the optimal-action selection behavior over the two-pile state space. The learned policy closely matches the theoretical structure of optimal Nim play, achieving an optimal-action selection accuracy of approximately 93.3%. In particular, the diagonal states corresponding to zero nim-sum positions are identified consistently as losing states, while the remaining regions are classified largely in accordance with the theoretical optimal strategy.

Figure 5(c) shows the gameplay evaluation performance across different initial problem sizes. The learned policy maintained nearly perfect gameplay performance across all tested settings, with win rates remaining close to 100% even as the number of piles increased. These results suggest that the proposed framework is able to recover the underlying combinatorial structure of optimal Nim play while maintaining strong gameplay performance.

5 CONCLUSION

This paper studied Bayesian reinforcement learning for the n -pile Nim game using a random-walk Metropolis MCMC framework for posterior inference over policy parameters. Both theoretical analysis and experimental results showed that the proposed framework can recover the combinatorial structure underlying optimal Nim play while achieving near-optimal gameplay performance across multiple problem settings.

However, there are several limitations to this work. First, the computational cost of the MCMC-based framework increases substantially as the number of piles grows, potentially limiting scalability in more complex game settings. Second, the experiments focused primarily on linear feature-based policies and gameplay against relatively simple opponents, which may not fully capture the framework’s behavior in more realistic or highly competitive environments. In addition, while Nim provides a mathematically structured setting for studying interpretable reinforcement learning, it remains an open question whether similar probabilistic learning behavior generalizes to more complex combinatorial games such as Chess, Go, or Poker. Future work may investigate more scalable posterior inference methods, more diverse gameplay environments and opponent strategies, and extensions to more

expressive deep learning architectures to further study the interpretability and strategic behavior of Bayesian reinforcement learning systems in combinatorial games.

STATEMENT ON THE USE OF LLMs

ChatGPT was used as a writing assistance tool to improve clarity, revise wording, assist with $\text{\LaTeX}$ formatting, and support parts of the implementation and computational optimization process.

The ideas, problem formulation, theoretical development, and overall structure of the project reflect the author’s original work. LLM usage was limited to editing and implementation support rather than generating the core content.

REPRODUCIBILITY STATEMENT

The experiments in this paper were implemented in Python using NumPy and PyTorch and executed in a Google Colab environment with GPU acceleration. The paper provides the mathematical definitions, model formulation, posterior inference procedure, experimental settings, evaluation metrics, and baseline methods used throughout the study. Additional implementation details, feature-ablation settings, prior-distribution configurations, and experimental setups are provided in Appendix B.

REFERENCES

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.
- [2] Michael Albert, Richard Nowakowski, and David Wolfe. *Lessons in play: an introduction to combinatorial game theory*. AK Peters/CRC Press, 2019.
- [3] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [4] Charles L Bouton. Nim, a game with a complete mathematical theory. *Annals of mathematics*, 3(1/4):35–39, 1901.
- [5] Alexis Conneau, German Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. What you can cram into a single vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126–2136, 2018.
- [6] Alexander D’Amour, Katherine Heller, Dan Moldovan, Ben Adlam, Babak Alipanahi, Alex Beutel, Christina Chen, Jonathan Deaton, Jacob Eisenstein, Matthew D Hoffman, et al. Underspecification presents challenges for credibility in modern machine learning. *Journal of Machine Learning Research*, 23(226):1–61, 2022.
- [7] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.
- [8] Mohammad Ghavamzadeh, Shie Mannor, Joelle Pineau, and Aviv Tamar. Bayesian reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 8(5-6):359–483, 2015.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [10] Michael I Jordan, Zoubin Ghahramani, Tommi S Jaakkola, and Lawrence K Saul. An introduction to variational methods for graphical models. *Machine learning*, 37(2):183–233, 1999.

-
- [11] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
 - [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
 - [13] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
 - [14] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017.
 - [15] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
 - [16] Kevin P Murphy. *Machine learning: a probabilistic perspective*. MIT press, 2012.
 - [17] Christian P Robert, George Casella, and George Casella. *Monte Carlo statistical methods*, volume 2. Springer, 2004.
 - [18] Kenneth H Rosen. *Discrete mathematics & applications*. McGraw-Hill, 1999.
 - [19] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence*, 1(5):206–215, 2019.
 - [20] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
 - [21] Adrian Smith. *Sequential Monte Carlo methods in practice*. Springer Science & Business Media, 2013.
 - [22] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR, 2017.
 - [23] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
 - [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
 - [25] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pages 818–833. Springer, 2014.

A DETAILED PROOFS

A.1 PROOF OF THEOREM 2.8

Proof. We prove both directions.

($\Rightarrow$) Suppose $x_1 \oplus \cdots \oplus x_n = 0$. Let s' be any state obtained from s by a legal move. Then for some i , the value x_i is replaced by y_i with $0 \leq y_i < x_i$. The nim-sum of s' is

$$x_1 \oplus \cdots \oplus x_{i-1} \oplus y_i \oplus x_{i+1} \oplus \cdots \oplus x_n.$$

If this were equal to 0, then applying $\oplus (x_1 \oplus \cdots \oplus x_{i-1} \oplus x_{i+1} \oplus \cdots \oplus x_n)$ to both sides gives $y_i = x_i$, which is a contradiction. Hence, every legal move from s leads to a state with a nonzero nim-sum, so s is losing.

($\Leftarrow$) Suppose $x_1 \oplus \cdots \oplus x_n \neq 0$. Let $m = x_1 \oplus \cdots \oplus x_n$.

Let k be the largest index such that the k -th binary digit of m is 1. Then there exists an index i such that the k -th binary digit of x_i is also 1. Define $y_i = x_i \oplus m$. Then $y_i < x_i$, so replacing x_i by y_i is a legal move. The nim-sum of the new state is

$$(x_1 \oplus \cdots \oplus x_n) \oplus x_i \oplus y_i = m \oplus x_i \oplus (x_i \oplus m) = 0.$$

Thus there exists a move to a state with nim-sum 0, so s is winning. $\square$

A.2 PROOF OF PROPOSITION 3.1

Proof. Let $q(\theta, \theta')$ denote the proposal density from θ to θ' . Since $\theta' = \theta + \eta$ and $\eta \sim N(0, \tau^2 I_d)$, the proposal density is

$$q(\theta, \theta') = \frac{1}{(2\pi\tau^2)^{d/2}} e^{-\frac{\|\theta' - \theta\|^2}{2\tau^2}} = \frac{1}{(2\pi\tau^2)^{d/2}} e^{-\frac{\|\theta - \theta'\|^2}{2\tau^2}} = q(\theta', \theta).$$

Therefore, the proposal is symmetric.

The Metropolis acceptance probability is

$$\alpha(\theta, \theta') = \min \left\{ 1, \frac{p(\theta' | D)}{p(\theta | D)} \right\}.$$

Hence, for $\theta \neq \theta'$, the transition kernel is

$$K(\theta, \theta') = q(\theta, \theta') \alpha(\theta, \theta').$$

To show that $p(\theta | D)$ is invariant, it suffices to verify the detailed balance condition

$$p(\theta | D) K(\theta, \theta') = p(\theta' | D) K(\theta', \theta).$$

Using the definition of K and the symmetry of q , we obtain

$$p(\theta | D) K(\theta, \theta') = p(\theta | D) q(\theta, \theta') \min \left\{ 1, \frac{p(\theta' | D)}{p(\theta | D)} \right\}.$$

We have the elementary identity

$$a \min \left\{ 1, \frac{b}{a} \right\} = \min\{a, b\}, \quad a, b > 0. \quad (1)$$

Applying (1) with $a = p(\theta | D)$, $b = p(\theta' | D)$ gives

$$p(\theta | D) K(\theta, \theta') = q(\theta, \theta') \min\{p(\theta | D), p(\theta' | D)\}.$$

Since $q(\theta, \theta') = q(\theta', \theta)$ and $\min\{p(\theta | D), p(\theta' | D)\} = \min\{p(\theta' | D), p(\theta | D)\}$, we have

$$p(\theta | D) K(\theta, \theta') = q(\theta', \theta) \min\{p(\theta' | D), p(\theta | D)\}.$$

Applying the same identity in reverse, we also obtain

$$q(\theta', \theta) \min\{p(\theta' | D), p(\theta | D)\} = p(\theta' | D) q(\theta', \theta) \min \left\{ 1, \frac{p(\theta | D)}{p(\theta' | D)} \right\}.$$

Therefore,

$$p(\theta | D) K(\theta, \theta') = p(\theta' | D) K(\theta', \theta).$$

Thus, detailed balance holds.

Finally, integrating both sides with respect to θ gives

$$\int p(\theta | D) K(\theta, \theta') d\theta = p(\theta' | D) \int K(\theta', \theta) d\theta.$$

Since $K(\theta', \cdot)$ is a transition kernel,

$$\int K(\theta', \theta) d\theta = 1.$$

Therefore,

$$\int p(\theta | D) K(\theta, \theta') d\theta = p(\theta' | D).$$

Hence, $p(\theta | D)$ is an invariant distribution for the Markov chain $\{\theta^{(t)}\}_{t \geq 0}$. $\square$

A.3 PROOF OF PROPOSITION 3.2

Proof. Let $\theta \in \mathbb{R}^d$ be any current state, and let $U \subseteq \mathbb{R}^d$ be any nonempty open set. We show that the probability of moving from θ into U in one step is positive.

The proposal density is

$$q(\theta, \theta') = \frac{1}{(2\pi\tau^2)^{d/2}} e^{-\frac{\|\theta' - \theta\|^2}{2\tau^2}},$$

Therefore,

$$q(\theta, \theta') > 0 \quad \text{for all } \theta, \theta' \in \mathbb{R}^d.$$

Since $p(\theta | D) > 0$ and $p(\theta' | D) > 0$, the acceptance probability

$$\alpha(\theta, \theta') = \min \left\{ 1, \frac{p(\theta' | D)}{p(\theta | D)} \right\} > 0 \quad \text{for all } \theta, \theta' \in \mathbb{R}^d.$$

Hence

$$q(\theta, \theta') \alpha(\theta, \theta') > 0 \quad \text{for all } \theta' \in U.$$

Therefore, the probability of proposing a point in U and accepting it is positive. Thus, the chain can move from any state θ to any nonempty open set U with positive probability. Hence, the chain is irreducible. $\square$

A.4 PROOF OF PROPOSITION 3.3

Proof. It is enough to show that from each state θ , there is positive probability that the chain stays at θ after one step.

Fix $\theta \in \mathbb{R}^d$. The chain stays at θ whenever a proposal is rejected. Thus $P(\theta, \theta) > 0$ provided that there is a set of proposed points θ' for which $\alpha(\theta, \theta') < 1$.

Now if $p(\theta' | D) < p(\theta | D)$, then

$$\alpha(\theta, \theta') = \frac{p(\theta' | D)}{p(\theta | D)} < 1.$$

Since the Gaussian proposal density is positive everywhere, there is a positive probability of proposing such a point θ' . Therefore, there is a positive probability that the proposal is rejected, and so $P(\theta, \theta) > 0$. Hence, the chain has period 1, so it is aperiodic. $\square$

A.5 PROOF OF COROLLARY 3.5

Proof. For each $i \in \{1, \dots, d\}$, applying Theorem 3.4 to $h_i(\theta) = \theta_i$ gives

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \theta_i^{(t)} = \int_{\mathbb{R}^d} \theta_i p(\theta | D) d\theta.$$

Hence,

$$\begin{aligned} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \theta^{(t)} &= \lim_{T \rightarrow \infty} \left(\frac{1}{T} \sum_{t=1}^T \theta_1^{(t)}, \dots, \frac{1}{T} \sum_{t=1}^T \theta_d^{(t)} \right) \\ &= \left(\int_{\mathbb{R}^d} \theta_1 p(\theta | D) d\theta, \dots, \int_{\mathbb{R}^d} \theta_d p(\theta | D) d\theta \right) \\ &= \int_{\mathbb{R}^d} \theta p(\theta | D) d\theta. \end{aligned}$$

$\square$

B ADDITIONAL EXPERIMENTAL AND IMPLEMENTATION DETAILS

This appendix provides additional implementation details for the Bayesian reinforcement learning framework, including the feature representations, feature-ablation settings, prior-distribution configurations, and additional experimental configurations used throughout the experiments.

B.1 FEATURE REPRESENTATIONS

The framework uses a feature vector constructed from structural properties of the resulting state and the selected action. The implemented features are:

1. **Bias term:** Constant feature equal to 1.
2. **Exact nim-sum indicator:** Indicator that the resulting state satisfies

$$x'_1 \oplus x'_2 \oplus \dots \oplus x'_n = 0.$$

3. **Immediate-win indicator:** Indicator that the resulting state is terminal.
4. **Normalized reduction in total objects:** Ratio between the number of removed objects and the total number of objects before the move.
5. **Remove-two indicator:** Indicator that exactly two objects are removed by the action.
6. **Normalized move size:** Move size normalized relative to the selected pile size.
7. **Game-progress feature:** Normalized proportion of removed objects relative to the initial game configuration.
8. **Largest-pile indicator:** Indicator that the selected pile is one of the largest piles in the current state.
9. **Normalized nim-sum before move:** Normalized value of the nim-sum before applying the action.
10. **Normalized nim-sum after move:** Normalized value of the nim-sum after applying the action.

B.2 FEATURE-ABLATION CONFIGURATIONS

The experiments considered three different feature configurations:

1. **Full feature representation:** Uses all ten features listed above.
2. **Without exact nim-sum feature:** Removes the feature indicating whether the resulting state has nim-sum equal to zero while retaining the remaining features.
3. **Reduced feature subset without nim information:** Removes all nim-sum-related features and retains only more general gameplay-related features.

B.3 PRIOR-DISTRIBUTION CONFIGURATIONS

Several prior distributions over the parameter vector θ were evaluated:

1. Gaussian prior with mean 0 and standard deviation 2,
2. Gaussian prior with mean 50 and standard deviation 2,
3. Gaussian prior with mean -50 and standard deviation 2,
4. Gaussian prior with mean 0 and standard deviation 10,
5. Half-normal prior with standard deviation 2,
6. Truncated normal prior with mean 0, standard deviation 2, and truncation interval $[-5, 5]$.

B.4 MCMC AND TRAINING CONFIGURATION

The posterior distribution was approximated using random-walk Metropolis MCMC with Gaussian proposal distributions. The experimental configuration used:

- 4,000 gameplay episodes,
- 4,000 MCMC iterations,
- burn-in period of 1,000 iterations,
- thinning interval of 10,
- posterior-mean parameter estimation from retained samples,
- evaluation over 1,000 simulated games per configuration.

C ADDITIONAL EXPERIMENTAL RESULTS

This appendix provides additional experimental results and visualizations for the proposed Bayesian reinforcement learning framework, including detailed summary statistics, training diagnostics, feature-ablation results, and prior-distribution analyses that complement the main experimental discussion in the paper.

C.1 TRAINING DIAGNOSTICS AND SUMMARY STATISTICS

Figure 6 presents additional training diagnostics across all experimental configurations, including posterior sampling behavior, acceptance-rate trends, and runtime scaling as the number of piles increases.

Table 1 provides detailed summary statistics across all experimental configurations, including evaluation win rates, MCMC acceptance rates, computational runtime, number of observed transitions, and action-space sizes for different feature representations and prior distributions.

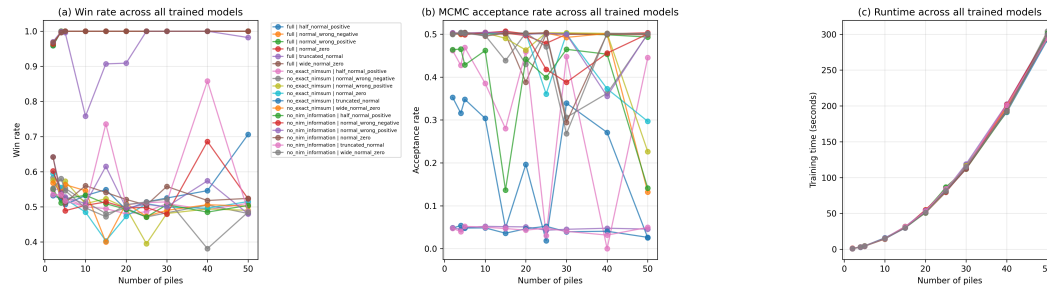


Figure 6: Additional training diagnostics across all experimental configurations.

n	Feature set	Prior distribution	Win rate	Acceptance	Time (s)	Transitions	Max actions
2	full	Half-normal ($\sigma = 2$)	0.9590	0.3524	1.0123	9103	20
4	full	Half-normal ($\sigma = 2$)	0.9990	0.3159	3.0803	17835	39
5	full	Half-normal ($\sigma = 2$)	1.0000	0.3479	4.6083	22284	49
10	full	Half-normal ($\sigma = 2$)	1.0000	0.3036	15.9326	44508	91
15	full	Half-normal ($\sigma = 2$)	1.0000	0.0479	30.0789	66362	118
20	full	Half-normal ($\sigma = 2$)	1.0000	0.1963	52.2520	88577	152
25	full	Half-normal ($\sigma = 2$)	1.0000	0.0187	80.7633	110996	184
30	full	Half-normal ($\sigma = 2$)	1.0000	0.3391	117.5364	133507	223
40	full	Half-normal ($\sigma = 2$)	1.0000	0.2708	191.2123	177993	274
50	full	Half-normal ($\sigma = 2$)	1.0000	0.0255	292.2394	222469	336
2	full	Gaussian ($\mu = -50, \sigma = 2$)	0.9630	0.5006	0.8872	8805	20
4	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.4996	2.9507	17755	38
5	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.5010	4.4926	22177	48
10	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.4999	14.7820	44624	84
15	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.5027	30.4809	66629	119
20	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.5021	55.0644	88708	158
25	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.5018	81.5754	111300	185
30	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.5011	115.6551	133286	220
40	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.5003	200.0100	177684	287
50	full	Gaussian ($\mu = -50, \sigma = 2$)	1.0000	0.1318	294.2368	222237	339
2	full	Gaussian ($\mu = 50, \sigma = 2$)	0.9590	0.4994	0.8908	8881	20
4	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5014	2.9549	17794	38
5	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5008	4.3270	22218	46
10	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5010	15.1901	44289	87
15	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5010	30.8955	66672	121
20	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5012	53.9390	88936	155
25	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5007	85.6321	111168	195
30	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.5007	112.5411	133266	214
40	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.4985	199.2869	177556	286
50	full	Gaussian ($\mu = 50, \sigma = 2$)	1.0000	0.4930	298.9659	222023	345
2	full	Gaussian ($\mu = 0, \sigma = 2$)	0.9640	0.5012	1.3860	8847	20
4	full	Gaussian ($\mu = 0, \sigma = 2$)	0.9970	0.5035	3.0062	17720	39
5	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.5015	4.5647	22187	49
10	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.5003	15.0683	44463	86
15	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.5024	30.7258	66748	120
20	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.4967	54.5955	88883	157
25	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.4785	82.5127	110989	188
30	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.4997	113.2882	133393	216
40	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.5004	198.7684	177591	285
50	full	Gaussian ($\mu = 0, \sigma = 2$)	1.0000	0.5031	300.6015	222469	346

Table 1: Experimental results across feature sets and prior distributions.

C.2 ADDITIONAL LEARNED-POLICY VISUALIZATIONS

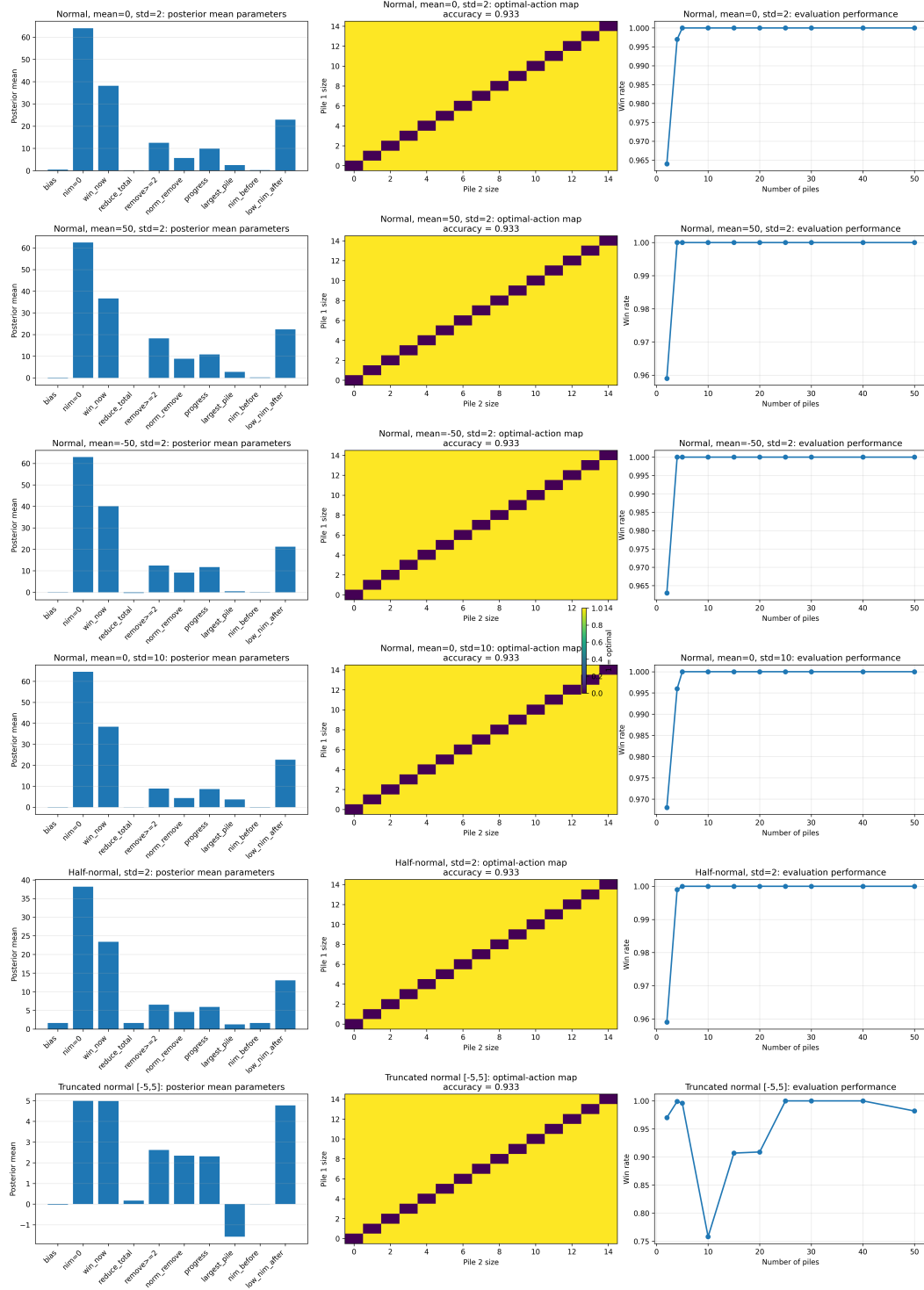


Figure 7: Learned-policy comparisons across different prior distributions.

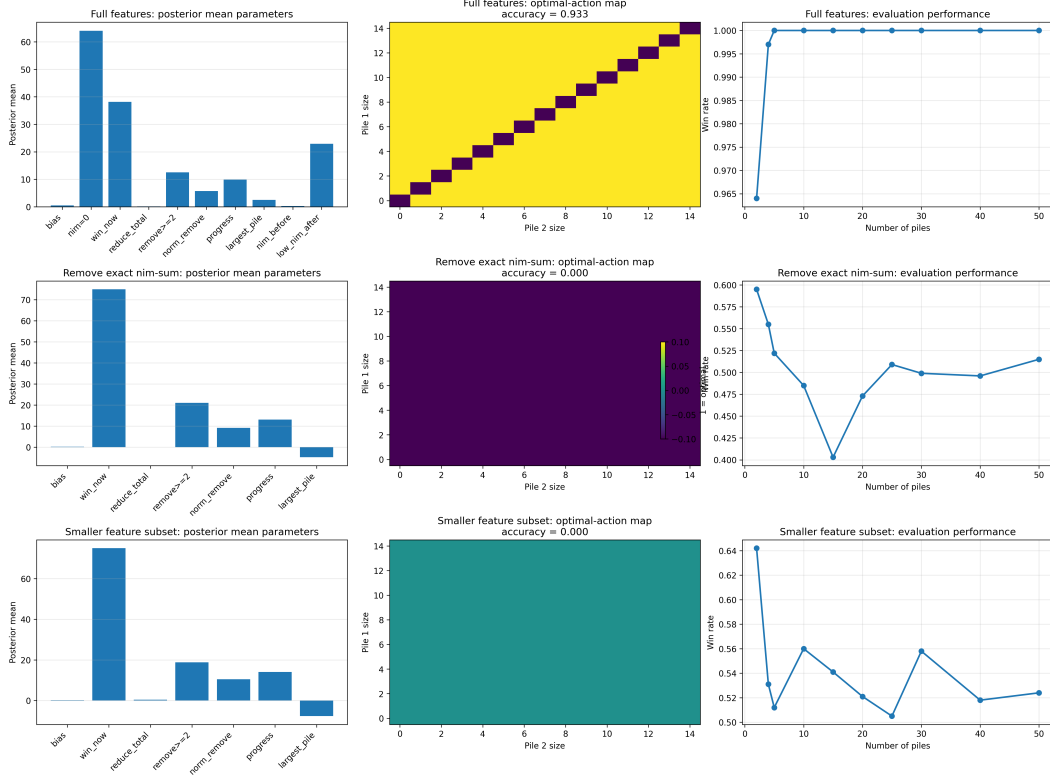


Figure 8: Learned-policy comparisons across different feature representations.

C.3 PER-CONFIGURATION POLICY PANELS

The following figures provide additional learned-policy visualizations for each feature-representation and prior-distribution configuration considered in the experiments.

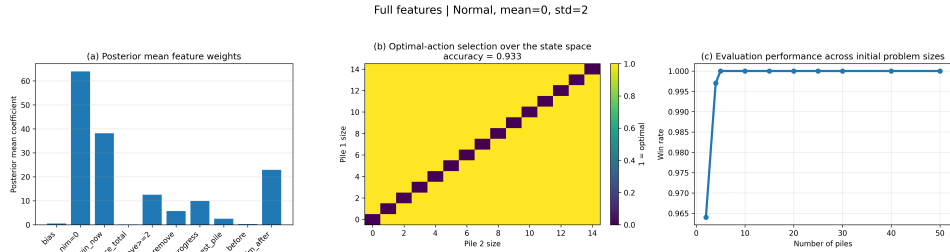


Figure 9: Full feature representation with Gaussian prior ($\mu = 0, \sigma = 2$).

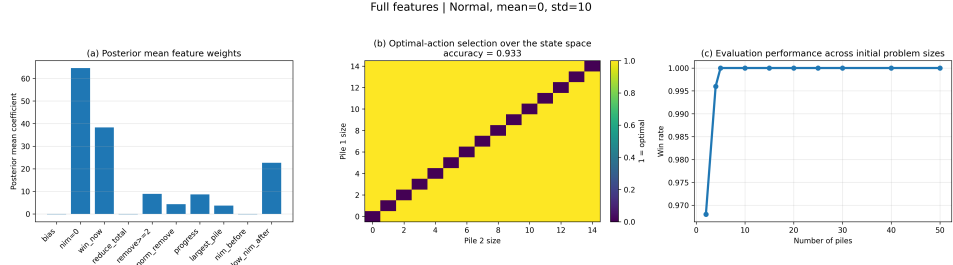


Figure 10: Full feature representation with Gaussian prior ($\mu = 0, \sigma = 10$).

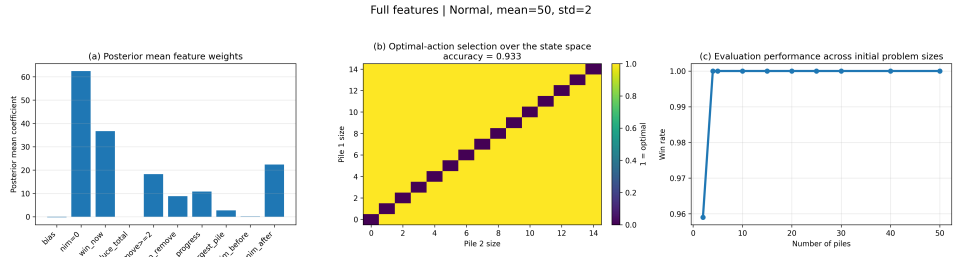


Figure 11: Full feature representation with Gaussian prior ($\mu = 50, \sigma = 2$).

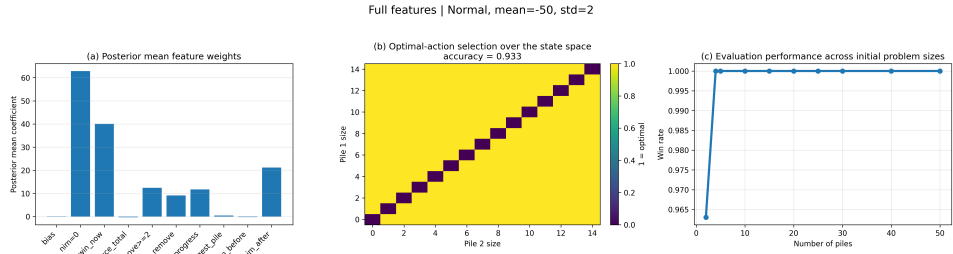


Figure 12: Full feature representation with Gaussian prior ($\mu = -50, \sigma = 2$).

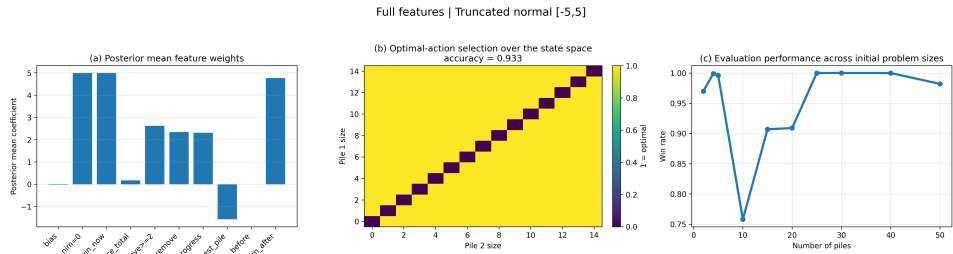


Figure 13: Full feature representation with truncated Gaussian prior ($\mu = 0, \sigma = 2$).

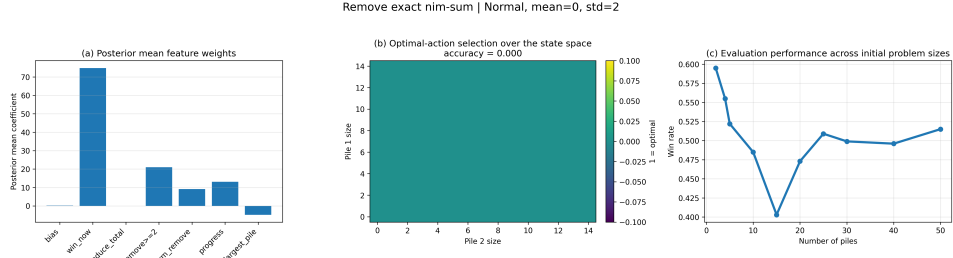


Figure 14: Feature representation without the exact nim-sum feature with Gaussian prior ($\mu = 0, \sigma = 2$).

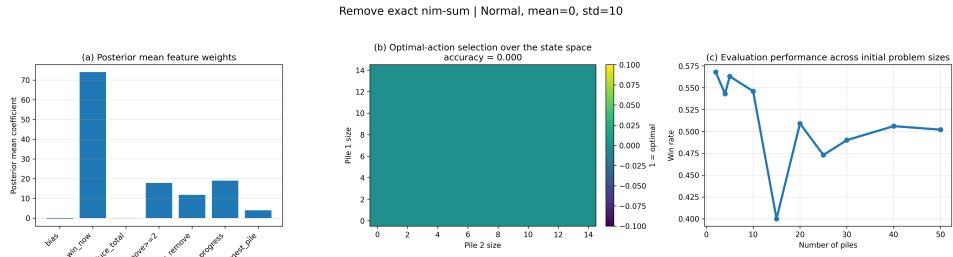


Figure 15: Feature representation without the exact nim-sum feature with Gaussian prior ($\mu = 0, \sigma = 10$).

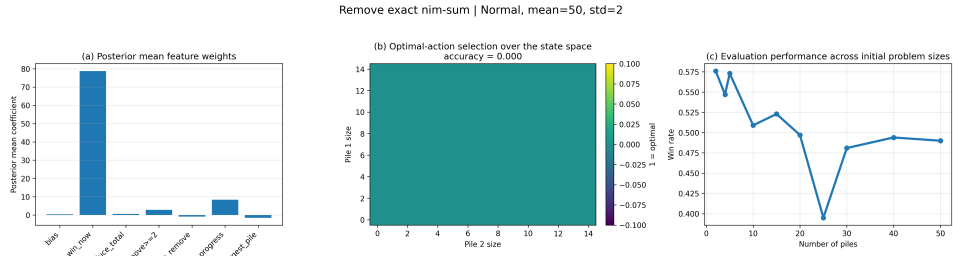


Figure 16: Feature representation without the exact nim-sum feature with Gaussian prior ($\mu = 50, \sigma = 2$).

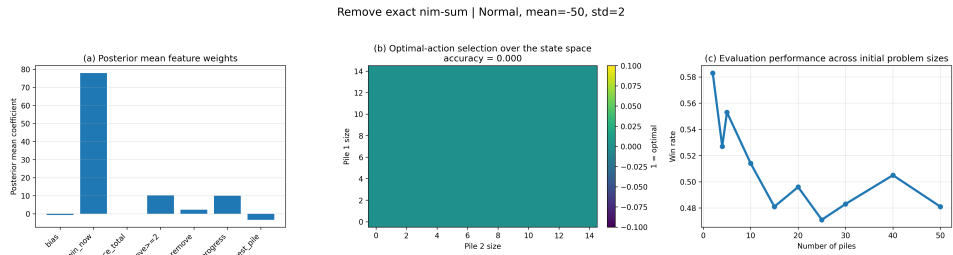


Figure 17: Feature representation without the exact nim-sum feature with Gaussian prior ($\mu = -50, \sigma = 2$).

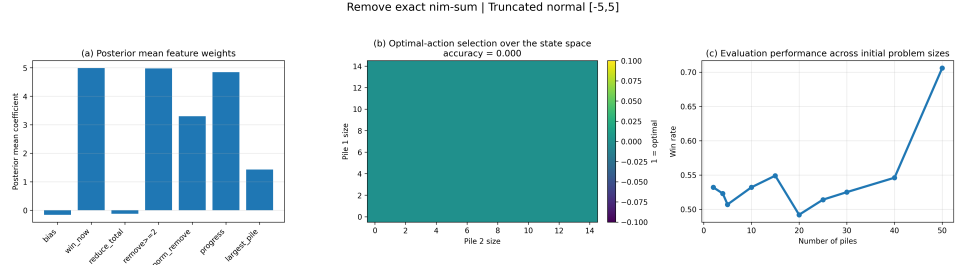


Figure 18: Feature representation without the exact nim-sum feature with truncated Gaussian prior ($\mu = 0, \sigma = 2$).

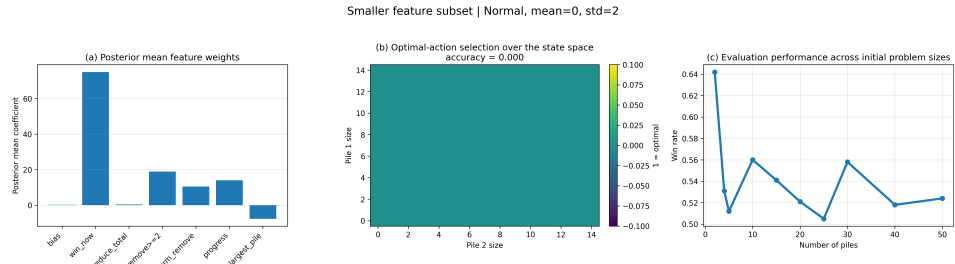


Figure 19: Reduced feature subset without nim-sum-related information with Gaussian prior ($\mu = 0, \sigma = 2$).

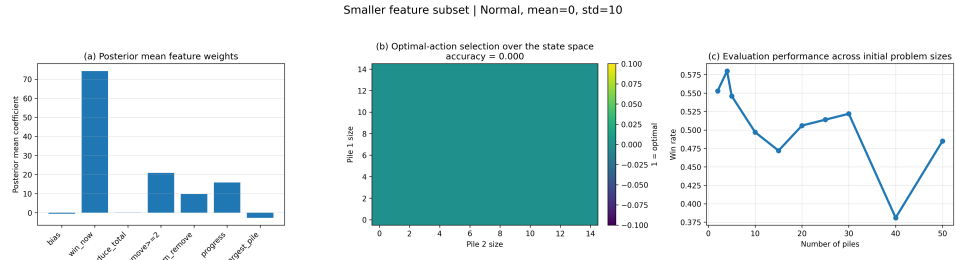


Figure 20: Reduced feature subset without nim-sum-related information with Gaussian prior ($\mu = 0, \sigma = 10$).

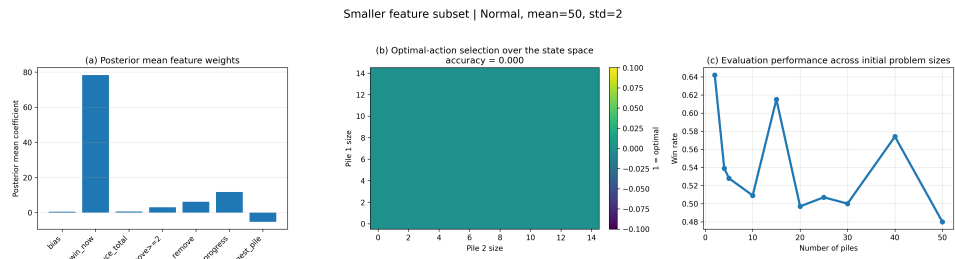


Figure 21: Reduced feature subset without nim-sum-related information with Gaussian prior ($\mu = 50, \sigma = 2$).

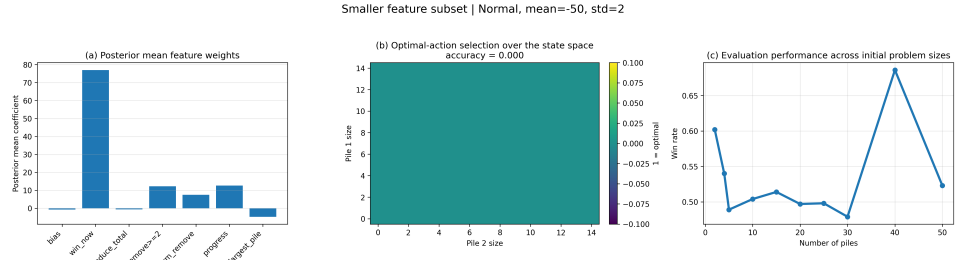


Figure 22: Reduced feature subset without nim-sum-related information with Gaussian prior ($\mu = -50, \sigma = 2$).

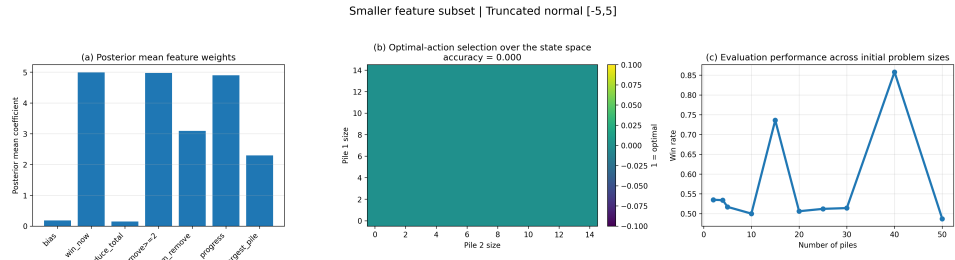


Figure 23: Reduced feature subset without nim-sum-related information with truncated Gaussian prior ($\mu = 0, \sigma = 2$).